Manual de ejecución

Asignación de Horarios Universitarios con GNU Prolog

1. Requisitos e instalación

El proyecto requiere Linux y GNU Prolog 1.4.5. En Windows, la ejecución se realiza desde una distribución de Linux instalada en WSL. Las herramientas necesarias son `gprolog`, `gplc`, un compilador de C y GNU Time.

En Ubuntu o Debian se instalan mediante:

```
sudo apt update
sudo apt install gprolog build-essential time
```

La versión instalada se consulta con:

```
gprolog --version
gplc --version
```

2. Ubicación del proyecto

Los comandos de este manual se ejecutan desde la carpeta que contiene `horarios.pl`, `horarios.sh`, `src` e `instances`. Se habilita el guion de envoltura y se crean las carpetas utilizadas en los ejemplos:

```
cd "<ruta-del-proyecto>"
chmod +x horarios.sh
mkdir -p output/experiments build
```

Las rutas relativas se interpretan respecto de esta ubicación. Las rutas con espacios deberán escribirse entre comillas en la terminal. La instancia, el reporte y la exportación deberán referirse a archivos diferentes, incluso si se utilizan rutas equivalentes con componentes como `./`.

El programa reemplaza los archivos de salida existentes. Sus carpetas deben permitir la escritura; no se crean automáticamente.

3. Formato de entrada

La instancia se describe en un archivo de texto plano con extensión `.dat`, codificado en UTF-8 y con saltos de línea de tipo Unix, LF. Cada registro ocupa una línea y sus campos se separan mediante punto y coma.

Los encabezados comienzan con `#:` y utilizan los nombres de las subsecciones siguientes. Cada encabezado aparece a lo sumo una vez y el orden de las secciones es indistinto. Los comentarios comienzan con `#`, salvo los encabezados; las líneas vacías se ignoran.

Los espacios al inicio y al final de los campos se eliminan. Los campos numéricos se escriben como enteros decimales no negativos, sin signo ni separador decimal. Las referencias deben coincidir con los identificadores declarados.

3.1. CONFIG

Registros de la forma `clave;valor`. Las cuatro claves son obligatorias y aparecen exactamente una vez.

Clave	Descripción	Ejemplo
`dias`	Códigos de día, de un carácter cada uno, separados por comas. Su orden define la cronología de los días.	`L,M,X,J,V`
`franjas`	Cantidad de franjas horarias por día, numeradas desde 1.	`7`
`inicio`	Hora de inicio de la primera franja, en formato `HH:MM`.	`07:00`
`duracion_franja`	Duración de cada franja, en minutos.	`60`

3.2. AULA

```
codigo;capacidad;tipo
```

El código es único y la capacidad indica el máximo de estudiantes que admite el aula. Las categorías de aula y de requerimiento de grupo son `teoria`, `laboratorio` y `mixta`.

3.3. PROFESOR

```
codigo;nombre;max_franjas;disponibilidad
```

El código es único. `max_franjas` establece la carga semanal máxima, considerando la duración de las sesiones. La disponibilidad se expresa mediante rangos del mismo día, con extremos inclusivos, como `L1-L4,M2-M5`. Un campo vacío indica disponibilidad total. Esta condición se comprueba sobre la franja inicial de cada sesión.

3.4. CURSO

```
codigo;nombre;grupo;inscritos;semanales;duracion;requiere
```

El par `(codigo, grupo)` es único. `inscritos` indica el número de estudiantes; `semanales`, la cantidad de sesiones por semana; `duracion`, las franjas consecutivas de cada sesión; y `requiere`, la categoría de aula necesaria.

3.5. IMPARTE

```
curso;grupo;profesor
```

Cada grupo debe tener al menos un profesor. Cuando se declaran varios, todos participan en cada sesión del grupo. La repetición de una relación no duplica sesiones ni profesores.

3.6. RESTRICCION

Se admiten las siguientes formas:

```
aula_fija;curso;grupo;aula
prohibida;curso;grupo;diaFranja
excluyentes;curso;grupo;otroCurso;otroGrupo
prefiere;curso;grupo;diaFranja;peso
compacta;curso;grupo;peso
```

Las restricciones duras son `aula_fija`, que exige utilizar el aula indicada; `prohibida`, que impide cubrir la franja indicada; y `excluyentes`, que impide el solapamiento de los dos grupos.

Las restricciones blandas son `prefiere`, que agrega el peso al costo si ninguna sesión del grupo comienza en la ubicación indicada, y `compacta`, que agrega el peso por cada franja libre entre la primera y la última franja ocupada por el grupo en un mismo día. El valor `L3`, por ejemplo, identifica la franja 3 del día `L`.

4. Ejecución desde la terminal

El guion `horarios.sh` invoca a GNU Prolog y carga el programa. La sintaxis es:

```
./horarios.sh <entrada.dat> <salida.txt> <estrategia> [opciones]
```

Los tres argumentos iniciales son obligatorios: ruta de la instancia, ruta del reporte y estrategia, respectivamente. Las estrategias admitidas son `bt` y `clpfd`. Los nombres de opciones y valores distinguen mayúsculas y minúsculas; cada opción se especifica a lo sumo una vez.

4.1. Opciones

Opción	Descripción
`-heuristica=VALOR`	Selecciona el criterio de exploración. El valor predeterminado es `original`.
`-optimizar`	Activa la búsqueda de menor costo mediante *branch and bound*.
`-limite=N`	Fija el máximo de nodos admitidos. `N` es un entero decimal no negativo; con cero no se admite el nodo inicial. Si se omite, no hay límite de nodos.
`-horarios-por-aula`	Agrega las rejillas de aulas al reporte.
`-exportar-prolog=RUTA`	Escribe un archivo Prolog adicional cuando existe un horario completo.

Las heurísticas admitidas por ambas estrategias son:

- `original`: selección de sesiones en el orden original.
- `mrv`: selección de la sesión con menos ubicaciones compatibles o viables.
- `degree`: selección de la sesión con mayor grado de restricciones con otras sesiones pendientes.
- `demand`: prioridad al mayor número de inscritos y, en caso de empate, a la mayor duración.
- `lcv`: selección original de sesiones y ordenamiento de ubicaciones según las opciones que dejan a las demás.

Se utiliza una sola heurística por ejecución. `ff` se admite únicamente con `clpfd` y aplica el mismo criterio que `mrv` en esta implementación.

4.2. Ejemplos

Los siguientes comandos muestran backtracking con exportación, backtracking con MRV, dominios finitos con `ff` y optimización con límite:

```
./horarios.sh instances/valid/small.dat output/small.txt bt \
    --horarios-por-aula --exportar-prolog=output/small.pl

./horarios.sh instances/valid/constraints.dat \
    output/constraints-mrv.txt bt --heuristica=mrv

./horarios.sh instances/valid/small.dat \
    output/small-fd.txt clpfd --heuristica=ff

./horarios.sh instances/valid/optimization.dat \
    output/optimization.txt clpfd \
    --optimizar --limite=5000000
```

5. Ejecución interactiva

El programa se carga mediante:

```
gprolog --consult-file horarios.pl
```

En el intérprete se utiliza `main/2`, con una lista de argumentos y una variable para recibir el código de terminación. Las opciones se agregan a la lista como átomos:

```
main(['instances/valid/optimization.dat',
    'output/interactive.txt', clpfd,
    '--optimizar', '--limite=5000000'], ExitCode).
```

El intérprete permanece abierto al finalizar la consulta. Para salir se utiliza `halt.`

6. Ejecución compilada

La compilación y la ejecución nativas utilizan:

```
gplc --new-top-level -o build/horarios horarios.pl

./build/horarios --entry-goal main -- \
    instances/valid/small.dat output/small-native.txt bt
```

La opción `-new-top-level` permite reconocer `-entry-goal`. Los argumentos posteriores a `--` corresponden a la interfaz de la sección 4. Si cambia el código fuente, deberá repetirse la compilación.

7. Interpretación de los resultados

Los mensajes generados están escritos en inglés. La terminal informa el tamaño estimado del espacio de búsqueda, la cantidad de sesiones requeridas y la ruta del reporte escrito. Los errores se comunican por la salida de error.

7.1. Reporte de texto

Cuando existe un horario, el reporte contiene:

1. **Encabezado**: instancia, estrategia, heurística, estado y fecha.
2. `ASSIGNMENT DETAIL`: una fila por sesión, ordenada por curso, grupo, día y franja inicial, con sus profesores, aula y franjas ocupadas.

3. **Rejillas**: horarios por grupo y profesor. Las celdas libres se representan mediante `-`.

4. `SUMMARY`: sesiones, franjas ocupadas, huecos, costo, nodos, retrocesos, tiempos de CPU y de pared, y tasa de exploración.

Sin optimización se entrega el primer horario válido y se calcula su costo. Con optimización, la conclusión `Optimal schedule. Search finished` indica que se demostró el menor costo al finalizar sin alcanzar el límite. Si la tasa de exploración no puede calcularse porque el tiempo de CPU registrado es cero, se informa `not calculable`.

En ausencia de un horario, el resumen se sustituye por `DIAGNOSTIC`. Este bloque informa las sesiones requeridas frente a las franjas disponibles, la mayor carga y la causa demostrada.

7.2. Estados y códigos de terminación

Estado	Código	Significado
`SOLUTION FOUND`	0	Se encontró y escribió un horario completo verificado.
`NO SOLUTION`	2	Se demostró que no existe un horario válido.
`LIMIT REACHED`	3	Se intentó superar el límite de nodos. Si existe un horario completo, se entrega el mejor encontrado sin garantizar optimalidad; de lo contrario, una solución puede existir.
`INVALID INSTANCE`	1	Se detectó un error sintáctico o de coherencia y se generó el reporte sin iniciar la búsqueda.
Error en terminal	1	Error de argumentos, lectura o escritura, o inconsistencia por construcción.

El código del proceso se consulta inmediatamente después de su ejecución:

```
echo $?
```

En los errores de escritura, este código prevalece sobre el resultado previo de la búsqueda. Las inconsistencias por construcción se informan en la terminal y detienen el flujo antes de escribir el reporte normal.

7.3. Exportación a Prolog

El archivo adicional declara un hecho `asignacion/7` por sesión. Sus argumentos son curso, grupo, lista de profesores, aula, día, franja inicial y duración. Los identificadores conservan sus bytes UTF-8 como listas de enteros; por ejemplo, `C1` se representa como `[67, 49]`.

La exportación del primer ejemplo puede cargarse y consultarse mediante:

```
$ gprolog --consult-file output/small.pl
| ?- asignacion(Course, Group, Teachers, Room, Day, Start, Duration).
```

Los símbolos `$` y `| ?-` representan los indicadores de la terminal y del intérprete; no forman parte de los comandos.

8. Instancias de prueba

Los archivos incluidos se encuentran bajo `instances`:

Archivo relativo a `instances`	Resultado esperado
`valid/small.dat`	Dos sesiones, cuatro franjas ocupadas y costo cero; no declara restricciones blandas.
`valid/constraints.dat`	Cinco sesiones, ocho franjas ocupadas y costo óptimo cero al optimizar.
`valid/optimization.dat`	Costo óptimo 5 al optimizar.
`invalid/syntax.dat`	Error de formato en un registro de aula.
`invalid/coherence.dat`	Varios errores de coherencia.
`unsatisfiable/load.dat`	Carga imposible de grupo y profesor, detectada antes de buscar.
`unsatisfiable/rooms.dat`	Ausencia de solución por restricciones implícitas.
`unsatisfiable/restrictions.dat`	Ausencia de solución por una declaración de grupos excluyentes.
`benchmark/large.dat`	Ochenta sesiones y ciento sesenta franjas ocupadas cuando se encuentra un horario completo.

9. Reproducción de experimentos

Las comparaciones deben conservar la instancia, el límite y la configuración de optimización. Cada ejecución utiliza nombres distintos para sus archivos de resultados.

GNU Time permite registrar el tiempo de pared y los tiempos de CPU de usuario y de sistema:

```
/usr/bin/time -f 'wall=%e user=%U system=%S exit=%x' \
    -o output/experiments/bt-original.time \
    ./horarios.sh instances/valid/constraints.dat \
    output/experiments/bt-original.txt bt \
    --heuristica=original --limite=50000
```

Para comparar heurísticas se cambia `-heuristica`; para comparar estrategias se cambia `bt` por `clpfd`; y para comparar ejecución interpretada y compilada se utiliza la invocación de la sección 6 dentro del mismo comando de medición.

La ejecución sin restricciones blandas utiliza `small.dat`. La ejecución optimizada utiliza `optimization.dat`. El caso de mayor tamaño utiliza `large.dat`.

El tiempo externo comprende el arranque del proceso y la escritura de archivos. El tiempo del reporte comprende la resolución hasta el final de la búsqueda y, cuando corresponde, las búsquedas adicionales de diagnóstico. Estas últimas no tienen límite de nodos y sus métricas se incluyen en el resultado informado.

9.1. Variante básica de backtracking

La selección de variante se encuentra en `src/solvers/bt/search.pl`. El hecho actual utiliza `forward`, correspondiente a verificación anticipada. Para comparar generación y prueba se sustituye ese valor por `basic`:

```
btSearchVariant(basic).
```

La variante básica conserva los órdenes originales de sesiones y ubicaciones y verifica las restricciones duras al completar la asignación. Debe mantenerse un solo hecho de selección. Al terminar la comparación se restaura `forward`; la recompilación se realiza conforme a la sección 6. Esta selección no dispone de una opción de línea de comandos.

10. Recursos y solución de errores

Ante un fallo se revisan el mensaje de la terminal y el diagnóstico, cuando exista. El análisis sintáctico informa el primer registro incorrecto; la validación de coherencia informa conjuntamente los errores encontrados. Un reporte conservado de una ejecución anterior no acredita el resultado actual.

Las búsquedas grandes, especialmente con MRV o LCV, pueden agotar la pila global. Su tamaño se ajusta mediante `GLOBALSZ`, expresado en KiB:

```
GLOBALSZ=262144 ./horarios.sh instances/benchmark/large.dat \
    output/large-fd.txt clpfd --heuristica=mrv --limite=50000
```

Este valor establece una pila global de 256 MiB. Debe elegirse según la memoria disponible; aumentarlo no garantiza que una búsqueda extensa termine. El límite de nodos no establece un límite de tiempo ni de memoria.

Los campos numéricos deben utilizar magnitudes representables por GNU Prolog. La versión actual no detecta el desbordamiento durante el parseo decimal.